

Slopes and Gradients

How a model knows which way to adjust · video 2:00

This lesson answers one question: when a model is wrong, **which way should each weight move?** The answer is the **slope**: how fast the output changes when you nudge the input. Take a small step against the slope, repeat, and you have **gradient descent**, the way LLMs are trained.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 9).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. What a slope measures	▶ 0:08-0:26	4	☐
2. The derivative: the slope at one point	▶ 0:26-0:40	4	☐
3. Step against the slope: gradient descent	▶ 0:41-1:06	4	☐
4. The gradient: one slope per weight	▶ 1:06-1:17	4	☐
5. Slopes multiply: the chain rule and backprop	▶ 1:17-1:40	4	☐
6. Gradient descent in code	▶ 1:40-1:45	3	☐

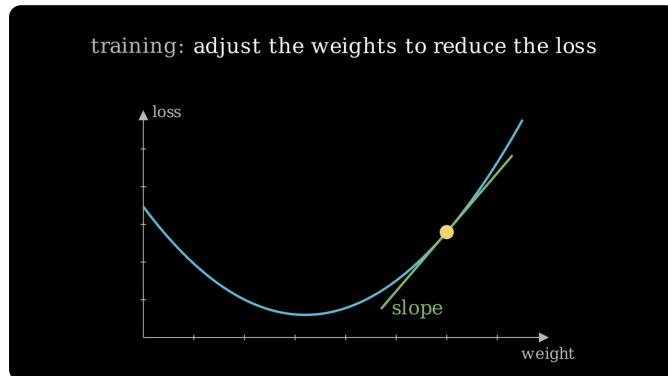
Before you start: this lesson builds on functions and their graphs from F04 (Straight Lines and Bends) and on F01 (Vectors), because the gradient is a vector. It is the math behind training in episode 11 (Training: Learning from Mistakes) and episode 12 (Build a Tiny GPT).

CONCEPT 1 OF 6

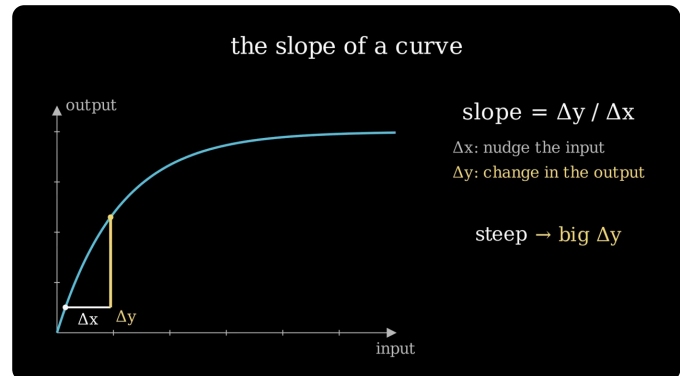
What a slope measures

▶ Video 0:08-0:26

Training adjusts a model's weights to reduce the **loss**, the number that says how badly the model is doing. To know **which way** to move each weight, we need one idea from calculus: the **slope**. It says how much the output changes when you nudge the input a little: **slope = $\Delta y / \Delta x$** , where Δx is the nudge and Δy the change it causes. Where the curve is **steep**, a small nudge gives a big change; where it is **flat**, almost none.



▶ 0:16 Which way should the weight move to lower the loss?



▶ 0:23 Steep: a small nudge Δx gives a big change Δy .

KEY IDEA

slope = $\Delta y / \Delta x$: the change in the output per unit of nudge. Steep means a big slope, flat a slope near 0. Turned around: a nudge Δx changes the output by about **slope $\times \Delta x$** .

Check 1: What a slope measures

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

You nudge the input from 2 to 2.5, and the output goes from 4 to 5.5. What is the slope?

.....

1.2 MULTIPLE CHOICE

Look at the curve in the second picture (steep on the left, flat on the right). Where does the same small nudge of the input change the output the most?

- A. On the right, where the curve is flat
- B. On the left, where the curve is steep
- C. Everywhere the same, because the nudge is the same size
- D. Nowhere: nudging the input never changes the output

1.3 TRUE OR FALSE

"Where the slope is close to 0, nudging the input a little barely changes the output."

True ☐ False ☐

1.4 CALCULATION

At the current weight, the loss curve has slope 2. You nudge the weight up by 0.01. About how much does the loss change, and does it go up or down?

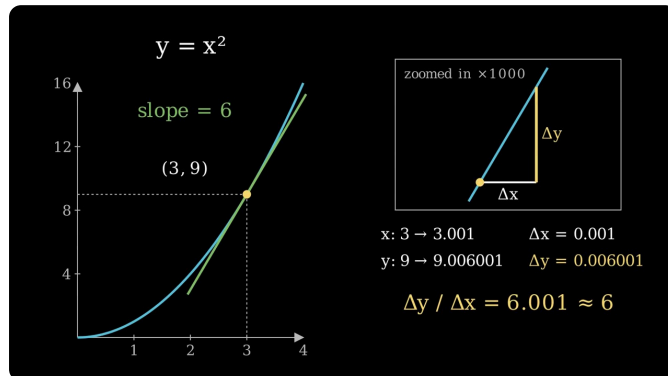
.....

CONCEPT 2 OF 6

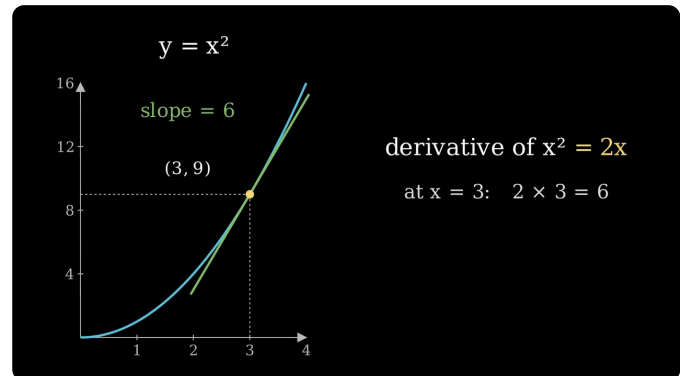
The derivative: the slope at one point

▶ Video 0:26-0:40

Take $y = x^2$. At $x = 3$ the output is 9. Nudge x by a tiny amount, from 3 to 3.001, and the output becomes 9.006001. So $\Delta y = 0.006001$ and $\Delta y / \Delta x = 6.001 \approx 6$: right there, the output grows about **six times as fast** as the input. The tinier the nudge, the closer the ratio gets to exactly 6.



▶ 0:36 Zoomed in 1000 times around $(3, 9)$: nudging x by 0.001 raises y by 0.006001, so $\Delta y / \Delta x \approx 6$.



▶ 0:39 The slope at $x = 3$ is 6. In general the derivative of x^2 is $2x$.

That rate is the **derivative**: the slope at a single point (the green line that just touches the curve). You don't have to nudge every time: for x^2 , the derivative at any x is $2x$. At $x = 3$ that is $2 \times 3 = 6$.

KEY IDEA

The **derivative** is the slope at one point: $\Delta y / \Delta x$ for a tiny nudge. **The derivative of x^2 is $2x$** , so the slope changes along the curve: 6 at $x = 3$, 0 at the bottom.

Check 2: The derivative: the slope at one point

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Use the rule $2x$. What is the slope of $y = x^2$ at $x = 5$? At $x = 0$? At $x = -2$?

.....

2.2 CALCULATION

Repeat the video's nudge at $x = 2$: compute 2.001^2 , then Δy and $\Delta y / \Delta x$. Which value does the rule $2x$ predict?

.....

.....

2.3 CALCULATION

Now use a bigger nudge at $x = 3$: from 3 to 3.1, so the output is $3.1^2 = 9.61$. What is $\Delta y / \Delta x$? Is it closer to 6 than 6.001, or further away?

.....

2.4 TRUE OR FALSE

"The slope of $y = x^2$ is 6 everywhere on the curve."

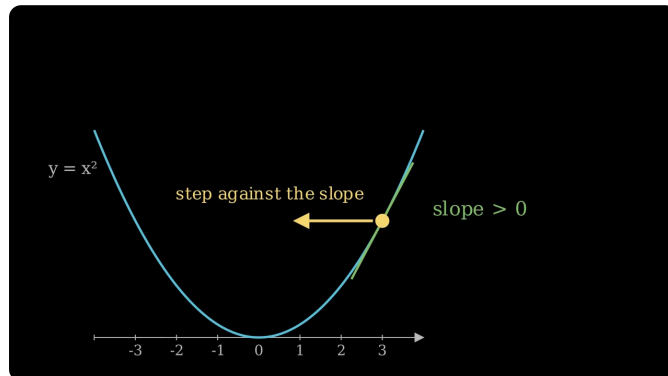
True ☐ False ☐

CONCEPT 3 OF 6

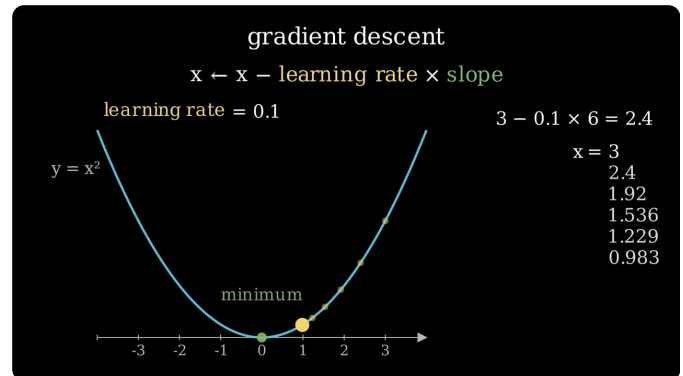
Step against the slope: gradient descent

▶ Video 0:41-1:06

The slope also tells you **which way is downhill**. At $x = 3$ on $y = x^2$ the slope is $+6$, so to make the output smaller we step left, **against the slope** (where the slope is negative, that means stepping right). That is **gradient descent: new $x = \text{old } x - \text{learning rate} \times \text{slope}$** , where the **learning rate** sets the step size. With 0.1 : $3 - 0.1 \times 6 = 2.4$. Repeat: $1.92, 1.536, 1.229, 0.983$, toward the minimum at 0 .



▶ 0:50 Slope > 0 at $x = 3$, so step left: against the slope.



▶ 1:05 Learning rate 0.1 : x slides toward the minimum at 0 .

KEY IDEA

$x \leftarrow x - \text{learning rate} \times \text{slope}$. Subtracting the slope always moves downhill, whatever its sign. The learning rate controls how big each step is.

Check 3: Step against the slope: gradient descent

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

Do the third step yourself: $x = 1.92$ on $y = x^2$, learning rate 0.1 . What is the slope there, and what is the new x ?

3.2 CALCULATION

Start instead at $x = -2$ (same curve, learning rate 0.1). What is the slope, where does one step take you, and which way did x move?

3.3 MULTIPLE CHOICE

At the current weight, the slope of the loss is -5 . To reduce the loss, you should:

- A. Decrease the weight
- B. Increase the weight
- C. Leave it alone: a negative slope means the loss is already at its minimum
- D. Set the weight to -5

3.4 PUT IN ORDER

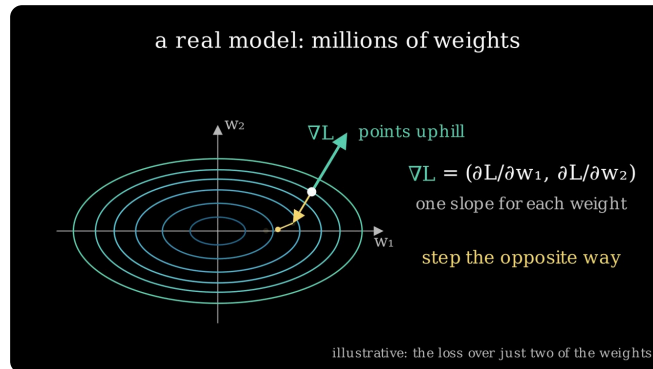
Put one round of gradient descent in order: *subtract the result from x · repeat from the new x · compute the slope at x · multiply the slope by the learning rate.*

CONCEPT 4 OF 6

The gradient: one slope per weight

▶ Video 1:06–1:17

A real model has millions of weights, not one, and each has its own slope: how much the loss changes when you nudge *that* weight alone, written $\partial L / \partial w_1$, $\partial L / \partial w_2$, ... The **gradient** ∇L collects one slope for each weight into a single **vector**. It points **uphill**, so we step the **opposite way**: every weight moves against its own slope, all at once.



▶ 1:16 Loss over two weights, seen from above. ∇L points uphill; the steps go the opposite way.

KEY IDEA

The **gradient** is a vector with **one slope per weight**. It points uphill, so each step goes the other way: $\mathbf{w} \leftarrow \mathbf{w} - \text{learning rate} \times \nabla L$.

Check 4: The gradient: one slope per weight

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

How many numbers are in the gradient of a model with 3 weights? With 1,000,000 weights?

4.2 CALCULATION

Two weights are at (2.4, 1.0). The slopes there are $\partial L / \partial w_1 = 2.4$ and $\partial L / \partial w_2 = 4.0$. Take one step with learning rate 0.2. Where do the weights go?

4.3 TRUE OR FALSE

“The gradient points downhill, so we add it to the weights.”

True ☐ False ☐

4.4 MULTIPLE CHOICE

The gradient for three weights is (0.5, -2.0, 0.0). With learning rate 0.1, which statement is right?

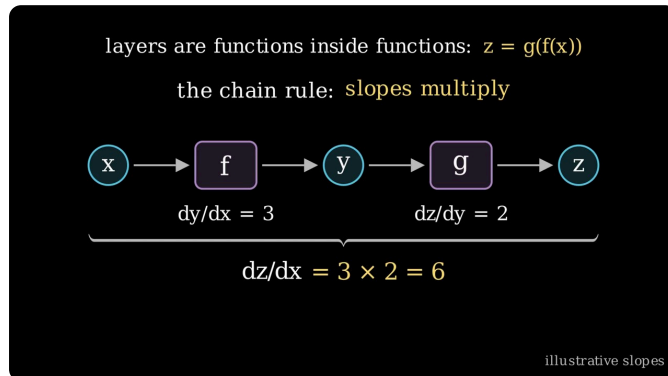
- A. w_3 changes the most, because its slope is 0
- B. w_2 is increased, and it moves the most of the three
- C. All three weights are decreased by the same amount
- D. w_1 is increased

CONCEPT 5 OF 6

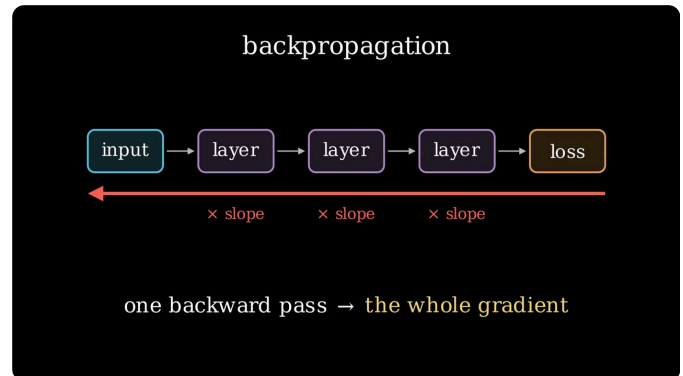
Slopes multiply: the chain rule and backprop

▶ Video 1:17-1:40

Layers are **functions inside functions**: $y = f(x)$, then $z = g(y)$. The **chain rule** says their slopes **multiply**: if y changes 3 times as fast as x ($dy/dx = 3$) and z twice as fast as y ($dz/dy = 2$), then z changes $3 \times 2 = 6$ times as fast as x . **Backpropagation** (backprop) applies this layer by layer, from the loss **backward** to every weight, so **one backward pass** gives the whole gradient.



▶ 1:31 Slopes multiply: $dz/dx = 3 \times 2 = 6$.



▶ 1:39 Backward from the loss, layer by layer.

KEY IDEA

Chain rule: $dz/dx = dy/dx \times dz/dy$. Backprop runs it from the loss backward: **one backward pass → the whole gradient**.

Check 5: Slopes multiply: the chain rule and backprop

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

Three layers in a row have slopes 2, 0.5 and 4. How many times as fast as the input does the final output change?

5.2 CALCULATION

Now $y = x^2$ and then $z = 5y$. At $x = 3$, find dy/dx , dz/dy and dz/dx .

5.3 TRUE OR FALSE

“Backpropagation needs one separate backward pass for each weight.”

True ☐ False ☐

5.4 MULTIPLE CHOICE

A network runs input → layer A → layer B → loss. Which layer does backprop reach first?

- A. Layer A, following the data
- B. Layer B: it starts at the loss and works backward
- C. Both at once, each on its own
- D. Neither: backprop only looks at the loss itself

CONCEPT 6 OF 6

Gradient descent in code

▶ Video 1:40-1:45

In code, gradient descent is a short loop: **compute the slope, step against it**. Here the slope of x^2 is written as a function, and five steps from $x = 3$ print the numbers from concept 3.

```
1 slope = lambda x: 2 * x      # derivative of x ** 2
2
3 x, lr = 3.0, 0.1
4 for step in range(5):
5     x = x - lr * slope(x)     # step against the slope
6     print(round(x, 3))        # 2.4, 1.92, 1.536, 1.229, 0.983
```

▶ 1:44 The code from the video: compute the slope, step against it.

```
slope = lambda x: 2 * x      # derivative of x ** 2

x, lr = 3.0, 0.1
for step in range(5):
    x = x - lr * slope(x)     # step against the slope
    print(round(x, 3))        # 2.4, 1.92, 1.536, 1.229, 0.983
```

KEY IDEA

The whole method is one line inside a loop: $x = x - lr * \text{slope}(x)$. Training a real model does the same, with the gradient in place of the slope.

Check 6: Gradient descent in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

Show that each step of this loop turns x into exactly $0.8 \times x$. Then use it: what is x after 10 steps (3 decimals)?

.....

.....

6.2 TRUE OR FALSE

“Each step in the printout moves x by the same amount.”

True ☐ False ☐

6.3 HANDS-ON CODE

Try it yourself. This version measures a slope by nudging, like the video's zoom, and lets you change the learning rate. Predict each output, then run it. (a) What does line (a) print? (b) What does `descend(3.0, 0.5)` print, and why? (c) What does `descend(3.0, 1.1)` print? What goes wrong?

```
def f(x):
    return x ** 2

def slope_at(f, x, h=0.001):
    return (f(x + h) - f(x)) / h    # nudge x, measure the change

def descend(x, lr, steps=5):
    for step in range(steps):
        x = x - lr * 2 * x          # the slope of x ** 2 is 2x
        print(round(x, 3), end=" ")
    print()

print(round(slope_at(f, 3), 3))    # (a)
descend(3.0, 0.5)                  # (b)
descend(3.0, 1.1)                  # (c)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. What a slope measures · rewatch ▶ 0:08-0:26

1.1 3. $\Delta x = 0.5$ and $\Delta y = 1.5$, so $\Delta y / \Delta x = 1.5 / 0.5 = 3$: the output changes three times as fast as the input.

1.2 B. The steeper the curve, the bigger Δy for the same Δx . On the flat part, Δy is tiny.

1.3 True. A slope near 0 is a flat curve: $\Delta y \approx \text{slope} \times \Delta x$ is almost nothing.

1.4 About 0.02, up. $\text{Change} \approx \text{slope} \times \text{nudge} = 2 \times 0.01 = 0.02$. The slope is positive, so moving the weight up moves the loss up.

2. The derivative: the slope at one point · rewatch ▶ 0:26-0:40

2.1 10, 0 and -4. $2 \times 5 = 10$; at 0 the curve is flat (the bottom of the bowl); at -2 the slope is negative: moving right, the curve goes down.

2.2 $2.001^2 = 4.004001$, $\Delta y = 0.004001$, $\Delta y / \Delta x = 4.001$. The rule predicts 4. $2 \times 2 = 4$. The tiny leftover 0.001 is there because the nudge is small but not zero.

2.3 6.1, further away. $\Delta y = 0.61$ and $0.61 / 0.1 = 6.1$. The curve bends, so a big nudge measures an average over a wide stretch. Tiny nudges give the slope at the point.

2.4 False. The slope is $2x$, so it depends on where you are: 6 at $x = 3$, 10 at $x = 5$, 0 at $x = 0$. Only a straight line has the same slope everywhere.

3. Step against the slope: gradient descent · rewatch ▶ 0:41-1:06

3.1 Slope 3.84, new $x = 1.536$. $\text{Slope} = 2 \times 1.92 = 3.84$; $1.92 - 0.1 \times 3.84 = 1.92 - 0.384 = 1.536$, the next number in the video.

3.2 Slope -4, new $x = -1.6$: it moved right. $-2 - 0.1 \times (-4) = -2 + 0.4 = -1.6$. Minus a negative slope is a step to the right, again toward the minimum at 0.

3.3 B. A negative slope means the loss goes down to the right. The update $w - \text{lr} \times (-5) = w + 5 \times \text{lr}$ increases the weight.

3.4 compute the slope → multiply by the learning rate → subtract from x → repeat. Exactly the rule $x \leftarrow x - \text{learning rate} \times \text{slope}$, applied again and again.

4. The gradient: one slope per weight · rewatch ▶ 1:06-1:17

4.1 3 and 1,000,000. One slope per weight, so the gradient is as long as the list of weights.

4.2 (1.92, 0.2). Each weight steps against its own slope: $2.4 - 0.2 \times 2.4 = 1.92$ and $1.0 - 0.2 \times 4.0 = 0.2$. This is the first yellow step in the picture.

4.3 False. The gradient points uphill (toward more loss). We subtract it, stepping the opposite way.

4.4 B. The changes are $-0.1 \times (0.5, -2.0, 0.0) = (-0.05, +0.2, 0)$: w_2 goes up by 0.2, w_1 goes down a little, and w_3 (slope 0) does not move.

5. Slopes multiply: the chain rule and backprop · rewatch ► 1:17-1:40

5.1 4 times. Slopes multiply: $2 \times 0.5 \times 4 = 4$.

5.2 6, 5 and 30. $dy/dx = 2x = 6$; $z = 5y$ changes 5 times as fast as y ; $6 \times 5 = 30$. Check by nudging: $x = 3.001$ gives $z = 45.030005$, a change of 0.030005 , which is $\approx 30 \times 0.001$.

5.3 False. One backward pass from the loss gives the slopes of all the weights: the whole gradient.

5.4 B. Backprop starts at the loss and multiplies on each layer's slope as it moves back toward the input, so B (next to the loss) comes before A.

6. Gradient descent in code · rewatch ► 1:40-1:45

6.1 $x - 0.1 \times 2x = 0.8x$; after 10 steps $x = 3 \times 0.8^{10} \approx 0.322$. Every step keeps 80 % of x , so the path is 3, 2.4, 1.92, ... and $3 \times 0.8^{10} = 0.322$. It gets close to 0 but never quite reaches it.

6.2 False. The steps are 0.6, 0.48, 0.384, ...: they shrink because the slope $2x$ shrinks as x nears the minimum.

6.3 (a) 6.001 · (b) 0.0 0.0 0.0 0.0 0.0 · (c) -3.6 4.32 -5.184 6.221 -7.465 (a) The same nudge as the video: $(9.006001 - 9) / 0.001 = 6.001$. (b) $3 - 0.5 \times 6 = 0$: this learning rate jumps straight to the minimum, and at 0 the slope is 0, so x stays put. (c) Each step is $x - 2.2x = -1.2x$: x jumps over the minimum and lands further away every time. A learning rate that is too big makes gradient descent blow up instead of settling down.